1 Obstacle Avoidance

(c)

At SCP iteration i, the cost is already convex, so only the dynamics and obstacle constraints are replaced by affine approximations. The convex subproblem is

$$\begin{aligned}
\min_{s_{\cdot|t},u_{\cdot|t}} \quad & (s_{N|t}-s_{\text{goal}})^T P(s_{N|t}-s_{\text{goal}}) + \sum_{k=0}^{N-1}\Big[(s_{k|t}-s_{\text{goal}})^T Q(s_{k|t}-s_{\text{goal}}) + u_{k|t}^T R u_{k|t}\Big] \\
\text{s.t.} \quad & s_{0|t} = s(t) \\
& s_{k+1|t} = A^{(i)}_{f,k|t} s_{k|t} + B^{(i)}_{f,k|t} u_{k|t} + c^{(i)}_{f,k|t}, \quad k = 0,\dots,N-1 \\
& A^{(i)}_{d,k|t} s_{k|t} + c^{(i)}_{d,k|t} \succeq 0, \quad k = 0,\dots,N
\end{aligned}$$

where

$$\begin{aligned}
A^{(i)}_{f,k|t} &= \left.\frac{\partial f}{\partial s}\right|_{(s^{(i)}_{k|t},u^{(i)}_{k|t})} \\
B^{(i)}_{f,k|t} &= \left.\frac{\partial f}{\partial u}\right|_{(s^{(i)}_{k|t},u^{(i)}_{k|t})} \\
c^{(i)}_{f,k|t} &= f(s^{(i)}_{k|t},u^{(i)}_{k|t}) - A^{(i)}_{f,k|t} s^{(i)}_{k|t} - B^{(i)}_{f,k|t} u^{(i)}_{k|t} \\
A^{(i)}_{d,k|t} &= \left.\frac{\partial d}{\partial s}\right|_{s^{(i)}_{k|t}} \\
c^{(i)}_{d,k|t} &= d(s^{(i)}_{k|t}) - A^{(i)}_{d,k|t} s^{(i)}_{k|t}
\end{aligned}$$

(d)

Each signed distance $d_j(s)$ is convex, so its first-order Taylor approximation is a global lower bound:

(g)

N	N_{SCP}	total time (s)	total control cost
5	5	20.30	6.93
2	5	18.80	11.18
5	2	11.81	4.43
15	2	14.49	4.50

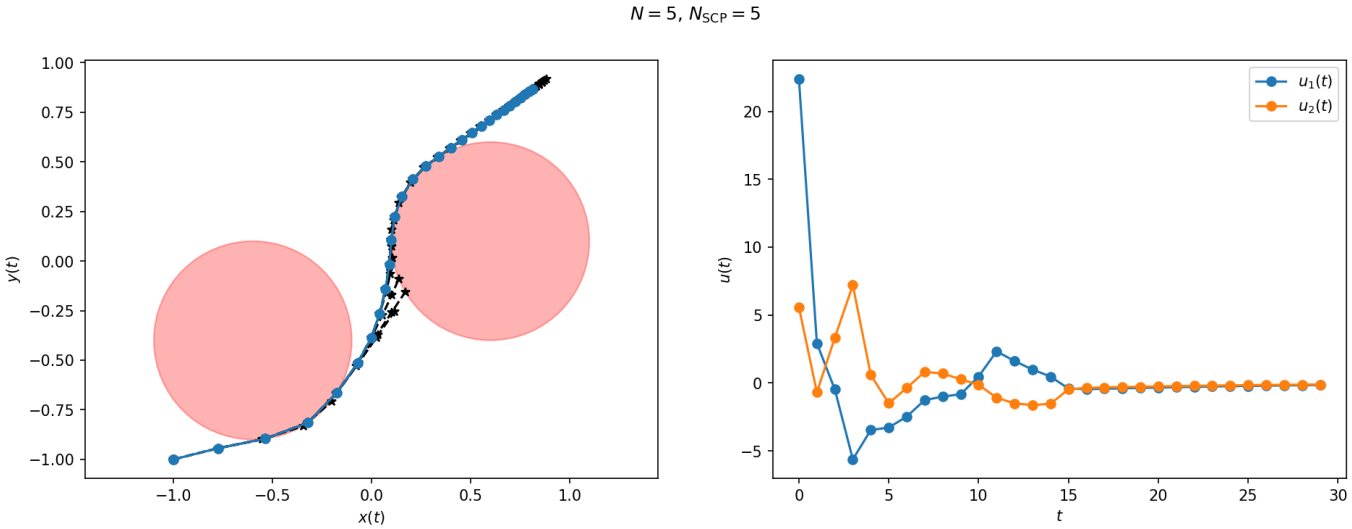

Figure 1: $N = 5,\ N_{\text{SCP}} = 5$

With $N = 5$ and $N_{\text{SCP}} = 5$, the trajectory avoids both obstacles and is reasonably smooth. This case takes the most time because each MPC solve uses more SCP iterations.

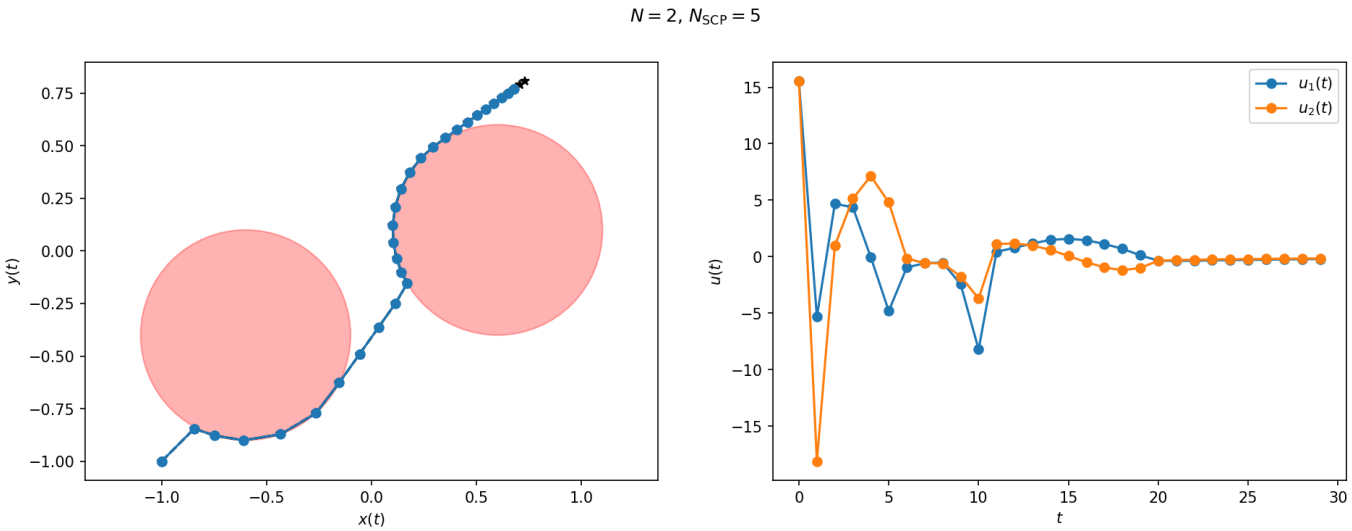

Figure 2: $N = 2,\ N_{\text{SCP}} = 5$

With $N = 2$, the controller is much more short-sighted. It still avoids the obstacles, but the path and controls are less smooth, and the total control cost is the largest.

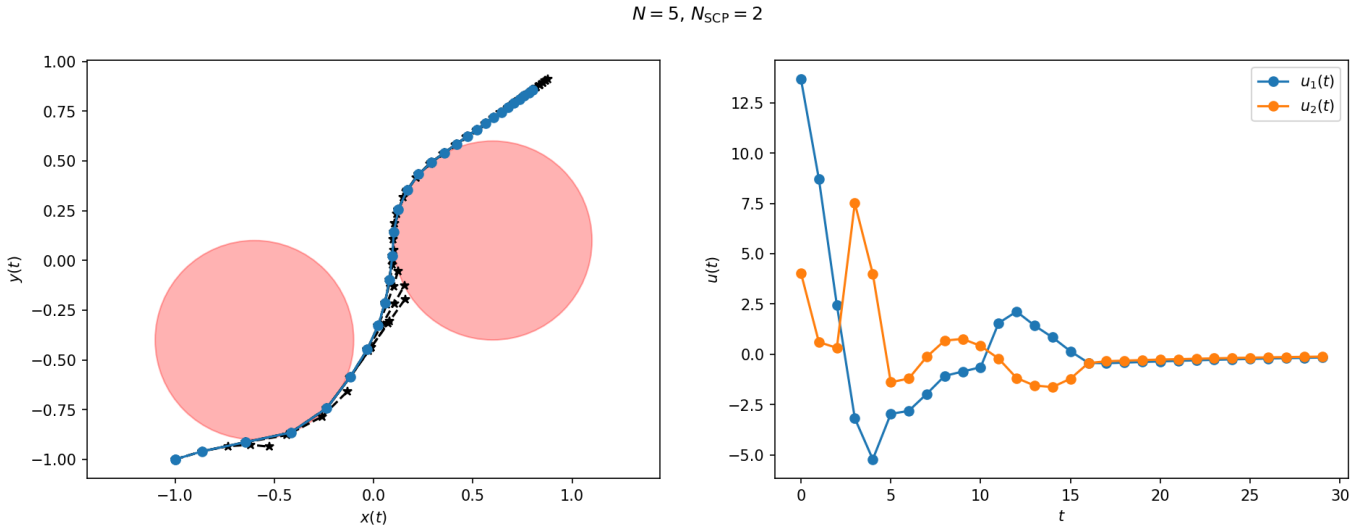

Figure 3: $N = 5,\ N_{\text{SCP}} = 2$

With $N = 5$ and only two SCP iterations, the computation time drops a lot while still giving a good trajectory. In this run it also gives the lowest control cost.

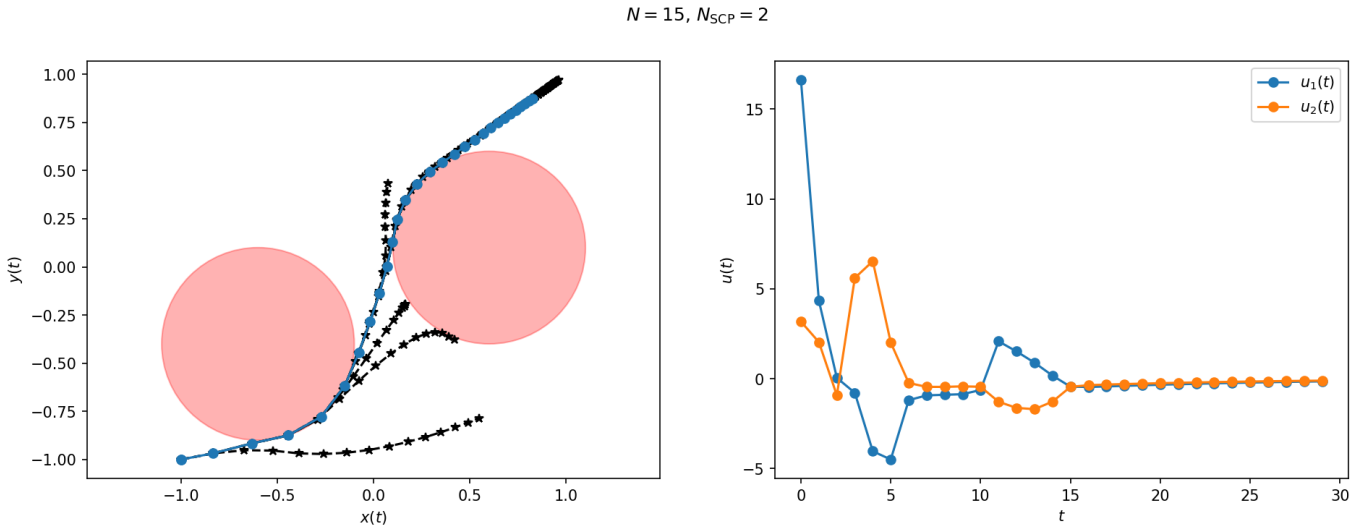

Figure 4: $N = 15,\ N_{\text{SCP}} = 2$

With $N = 15$, the planned trajectories are smoother because the controller can see farther ahead. The solve time is higher than $N = 5,\ N_{\text{SCP}} = 2$, but the control cost is still low.

Code Appendix

obstacle_avoidance.py

```
"""
Starter code for the problem "Obstacle avoidance".

Autonomous Systems Lab (ASL), Stanford University
"""

from functools import partial
from time import time

import cvxpy as cvx

import jax
import jax.numpy as jnp

import matplotlib.pyplot as plt

import numpy as np

from tqdm.auto import tqdm


def dynamics(s, u, beta=0.3, dt=0.1):
    """Evaluate the discrete-time dynamics of the system."""
    _, _, dx, dy = s
    ds = jnp.array(
        [
            dx,
            dy,
            u[0] - beta * dx * jnp.abs(dx),
            u[1] - beta * dy * jnp.abs(dy),
        ]
    )
    s_next = s + dt * ds
    return s_next


def signed_distances(s, centers, radii):
    """Compute signed distances to circular obstacles."""
    centers = jnp.reshape(centers, (-1, 2))
    radii = jnp.reshape(radii, (-1,))

    # PART (a): YOUR CODE BELOW ###################################################
    # INSTRUCTIONS: Compute the vector of signed distances to each obstacle.
    d = jnp.linalg.norm(s[:2] - centers, axis=1) - radii
    # END PART (a) ################################################################
    return d


@partial(jax.jit, static_argnums=(0,))
@partial(jax.vmap, in_axes=(None, 0, 0))
def affinize(f, s, u):
    """Affinize the function `f(s, u)` around `(s, u)`."""
    # PART (b) ####################################################################
    # INSTRUCTIONS: Use JAX to affinize `f` around `(s,u)` in two lines.
    A,B = jax.jacobian(f, argnums=(0,1))(s,u)
    c = f(s,u) - A @ s - B @ u
    # END PART (b) ################################################################
    return A, B, c
```

```python


def scp_iteration(f, d, s0, s_goal, s_prev, u_prev, P, Q, R):
    """Solve a single SCP sub-problem for the obstacle avoidance problem."""
    n = s_prev.shape[-1]  # state dimension
    m = u_prev.shape[-1]  # control dimension
    N = u_prev.shape[0]  # number of steps

    Af, Bf, cf = affinize(f, s_prev[:-1], u_prev)
    Af, Bf, cf = np.array(Af), np.array(Bf), np.array(cf)
    Ad, _, cd = affinize(
        lambda s, _: d(s), s_prev, jnp.concatenate((u_prev, u_prev[-1:]))
    )
    Ad, cd = np.array(Ad), np.array(cd)

    s_cvx = cvx.Variable((N + 1, n))
    u_cvx = cvx.Variable((N, m))

    # PART (e): YOUR CODE BELOW ##################################################
    # INSTRUCTIONS: Construct the convex SCP sub-problem.

    objective = 0.0
    constraints = [ s_cvx[0,:] == s0 ]

    for t in range(N):
        objective += cvx.quad_form(s_cvx[t,:] - s_goal,Q) + cvx.quad_form(u_cvx[t,:], R)
        constraints += [    Ad[t] @ s_cvx[t,:] + cd[t] >= 0,
                            s_cvx[t+1] == Af[t] @ s_cvx[t,:] + Bf[t] @ u_cvx[t,:] + cf[t]
                                ]

    objective += cvx.quad_form(s_cvx[N,:] - s_goal,P)
    constraints += [Ad[N] @ s_cvx[N, :] + cd[N] >= 0]

    # END PART (e) ###############################################################

    prob = cvx.Problem(cvx.Minimize(objective), constraints)
    prob.solve()
    if prob.status != "optimal":
        raise RuntimeError("SCP solve failed. Problem status: " + prob.status)
    s = s_cvx.value
    u = u_cvx.value
    J = prob.objective.value
    return s, u, J


def solve_obstacle_avoidance_scp(
    f,
    d,
    s0,
    s_goal,
    N,
    P,
    Q,
    R,
    eps,
    max_iters,
    s_init=None,
    u_init=None,
    convergence_error=False,
):
    """Solve the obstacle avoidance problem via SCP."""
    n = Q.shape[0]  # state dimension
    m = R.shape[0]  # control dimension
```

```
121
122     # Initialize trajectory
123     if s_init is None or u_init is None:
124         s = np.zeros((N + 1, n))
125         u = np.zeros((N, m))
126         s[0] = s0
127         for k in range(N):
128             s[k + 1] = f(s[k], u[k])
129     else:
130         s = np.copy(s_init)
131         u = np.copy(u_init)
132
133     # Do SCP until convergence or maximum number of iterations is reached
134     converged = False
135     J = np.zeros(max_iters + 1)
136     J[0] = np.inf
137     for i in range(max_iters):
138         s, u, J[i + 1] = scp_iteration(f, d, s0, s_goal, s, u, P, Q, R)
139         dJ = np.abs(J[i + 1] - J[i])
140         if dJ < eps:
141             converged = True
142             break
143     if not converged and convergence_error:
144         raise RuntimeError("SCP did not converge!")
145     return s, u
146
147
148 # Define constants
149 n = 4  # state dimension
150 m = 2  # control dimension
151 s0 = np.array([-1.0, -1.0, 0.0, 0.0])  # initial state
152 s_goal = np.array([1.0, 1.0, 0.0, 0.0])  # desired final state
153 T = 30  # total simulation time
154 P = 1e2 * np.eye(n)  # terminal state cost matrix
155 Q = 1e1 * np.eye(n)  # state cost matrix
156 R = 1e-2 * np.eye(m)  # control cost matrix
157 eps = 1e-3  # SCP convergence tolerance
158
159 # Set obstacle center points and radii
160 centers = np.array(
161     [
162         [-0.6, -0.4],
163         [0.6, 0.1],
164     ]
165 )
166 radii = np.array([0.5, 0.5])
167
168 # PART (g): CHANGE THE PARAMETERS BELOW ########################################
169
170 N = 15  # MPC horizon
171 N_scp = 2  # maximum number of SCP iterations
172
173 # END PART (g) #################################################################
174
175 f = dynamics
176 d = partial(signed_distances, centers=centers, radii=radii)
177 s_mpc = np.zeros((T, N + 1, n))
178 u_mpc = np.zeros((T, N, m))
179 s = np.copy(s0)
180 total_time = time()
181 total_control_cost = 0.0
182 s_init = None
183 u_init = None
```

```
for t in tqdm(range(T)):
    # PART (f): YOUR CODE BELOW ################################################

    # Solve the MPC problem at time `t`
    # Hint: You should call `solve_obstacle_avoidance_scp` here
    s_mpc[t], u_mpc[t] = solve_obstacle_avoidance_scp(f,d,s,s_goal,N,P,Q,R,eps,N_scp,
        s_init,u_init)

    # Push the state `s` forward in time with a closed-loop MPC input
    # Hint: You should call `f` here
    s = np.array(f(s,u_mpc[t,0]))

    # END PART (f) #############################################################

    # Accumulate the actual control cost
    total_control_cost += u_mpc[t, 0].T @ R @ u_mpc[t, 0]

    # Use this solution to warm-start the next iteration
    u_init = np.concatenate([u_mpc[t, 1:], u_mpc[t, -1:]])
    s_init = np.concatenate(
        [s_mpc[t, 1:], f(s_mpc[t, -1], u_mpc[t, -1]).reshape([1, -1])]
    )
total_time = time() - total_time
print("Total elapsed time:", total_time, "seconds")
print("Total control cost:", total_control_cost)

fig, ax = plt.subplots(1, 2, dpi=150, figsize=(15, 5))
fig.suptitle("$N = {}$, ".format(N) + r"$N_\mathrm{SCP} = " + "{}$".format(N_scp))

for pc, rc in zip(centers, radii):
    ax[0].add_patch(plt.Circle((pc[0], pc[1]), rc, color="r", alpha=0.3))
for t in range(T):
    ax[0].plot(s_mpc[t, :, 0], s_mpc[t, :, 1], "--*", color="k")
ax[0].plot(s_mpc[:, 0, 0], s_mpc[:, 0, 1], "-o")
ax[0].set_xlabel(r"$x(t)$")
ax[0].set_ylabel(r"$y(t)$")
ax[0].axis("equal")

ax[1].plot(u_mpc[:, 0, 0], "-o", label=r"$u_1(t)$")
ax[1].plot(u_mpc[:, 0, 1], "-o", label=r"$u_2(t)$")
ax[1].set_xlabel(r"$t$")
ax[1].set_ylabel(r"$u(t)$")
ax[1].legend()

suffix = "_N={}_Nscp={}".format(N, N_scp)
plt.savefig("soln_obstacle_avoidance" + suffix + ".png", bbox_inches="tight")
# plt.show()
```